

가능성을 기회로 바꾸고, 제품으로 가치를 전하는 메이커 김대정입니다.

기획자로 시작해 백엔드로 왔고, 지금은 아이디어를 제안한 AI 콘텐츠 제품의 백엔드·AI 실행부를 직접 만들고 핵심 화면은 coding agent로 완성해 월 1천만원 수준의 구독 매출이 발생하는 제품으로 운영합니다. 팀이 coding agent로 만들어도 같은 기준이 되도록 FastAPI 조직 표준과 evidence로 달는 QA 판정 규칙을 세웠고, 제품 기능을 대화로 제어하는 Agent를 설계·검증했습니다.

하이퍼노바에서 먼저 확인할 네 가지

01 팀이 같은 기준으로 만드는 AI 코드 검증 체계

자주 틀리는 경계를 기본값으로 제공하는 FastAPI 조직 표준 template과, 요구사항을 REQ로 쪼개 evidence로 달는 QA 판정 규칙(PASS/FAIL/UNKNOWN)을 세웠습니다. 실행 성공과 품질 통과를 분리하고 LLM judge 단독 승인은 금지했습니다.

02 하네스를 먼저 세운 배포

패턴·계층·검증 하네스를 먼저 세우고 그 위에서 coding agent와 새 backend를 나란히 만들어 응답을 비교한 뒤 전환했습니다. 운영 중인 제품을 멈추지 않고 backend를 교체해 결함 재발을 약 94% 줄였습니다.

03 Agent 설계와 컨텍스트 실측

판정 위치를 planner로 옮기고, 생성 파이프라인은 15노드 legacy 그래프를 단일 파이프라인으로 축소한 뒤 단일 agent 엔진과 실행 시간을 비교했습니다. 대화형 제어는 하나의 planner-executor로 두고 컨텍스트 압축·먹통 receipt·사람 승인 경계를 실측으로 검증했습니다.

04 실시간 음성 AI의 순서 정합

부분·완성·보정 전사를 같은 sequence로 묶어 늦게 도착한 보정이 다른 발화를 덮지 않게 했고, 중복 event는 task cancellation-debounce-retry-turn-state guard로 제어했습니다.

FastAPI에서 트랜잭션과 AsyncSession의 책임을 Service 계층에 두었습니다.

Router에서 받은 session을 모든 계층에 전달하던 반복에서 시작했습니다. 업무를 시작하는 Service method가 트랜잭션 범위를 선언하고, Repository는 현재 session을 찾아 SQL 실행에만 집중하도록 나눴습니다. 하나의 트랜잭션을 하나의 asyncio task와 AsyncSession이 사용하게 해 편의보다 데이터 가시성과 실패 복구를 먼저 예측할 수 있는 구조를 택했습니다.

이 버전에서 보는 지점

팀이 coding agent로 기능을 만들기 시작하면서 백엔드 경험이 적은 담당자도 구현에 참여했고, QA와 운영 준비 단계에서 사용량이나 동시 요청 조건에 따라 구조적인 문제가 자주 드러났습니다. 기능마다 완벽한 구조를 요구하는 대신 자주 틀리는 경계를 기본값으로 제공한 판단, Service가 transaction 정책을 선언하고 Repository는 현재 session만 resolve하게 한 구조, 그리고 요구사항을 REQ로 쪼개 evidence로 담는 QA 판정 규칙을 QA 팀원의 서포트를 받아 세우고 실행 성공과 품질 통과를 분리한 과정을 봅니다. propagation-isolation-CancelledError rollback-connection cleanup을 integration test로 고정한 template 위에서 신규 프로그램의 STG QA에서 같은 유형의 session-connection 문제가 재관측되지 않았습니다.

반복 인자를 없애는 데서 끝내지 않고 트랜잭션 책임을 코드로 고정했습니다.

문제 · 선택

왜 바꿨나	무엇을 기본값으로 뒀나
session 인자가 Router부터 Repository까지 업무와 무관하게 반복됐습니다.	Router-Service-Validator-Repository-Model의 책임과 의존 방향을 고정했습니다.
트랜잭션을 참여·분리·종결할 위치가 호출부마다 달라질 수 있었습니다.	Service가 트랜잭션을 시작하고 Repository는 SQL 실행과 flush만 담당합니다.
여러 직군이 Coding Agent로 구현해도 같은 구조를 따를 기본값이 필요했습니다.	FastAPI의 Depends를 부정하지 않고, 업무 단위 정책을 Service에 모았습니다.

중첩 호출에서 달라지는 동작과 실패 범위를 명시했습니다.

RUNTIME BEHAVIOR

구분	설계한 동작	실패·복구 범위
REQUIRED	기존 트랜잭션에 참여하고, 없으면 새로 시작	같은 session을 사용하며 바깥 트랜잭션과 함께 rollback
REQUIRES_NEW	바깥 트랜잭션을 멈추고 새 connection-AsyncSession에서 실행	독립적으로 commit-rollback한 뒤 바깥 작업을 재개
NESTED	같은 session-connection에 SAVEPOINT 생성	안쪽 작업만 rollback하고 바깥 트랜잭션은 계속 진행 가능
TASK GUARD	child task가 같은 AsyncSession에 접근하면 즉시 실패	task마다 트랜잭션·session·복구 방법을 따로 결정

Service가 정한 트랜잭션을 하위 계층이 반복 인자 없이 사용합니다.

REQUEST RESOURCE BINDING



session 인자 반복 없음 · 1 transaction · 1 task · 1 AsyncSession

내부 병렬성을 제한한 이유와 예외 처리 방법을 함께 남겼습니다.

TRADE-OFF · VERIFICATION

의도한 제약	검증한 실패 경로
--------	-----------

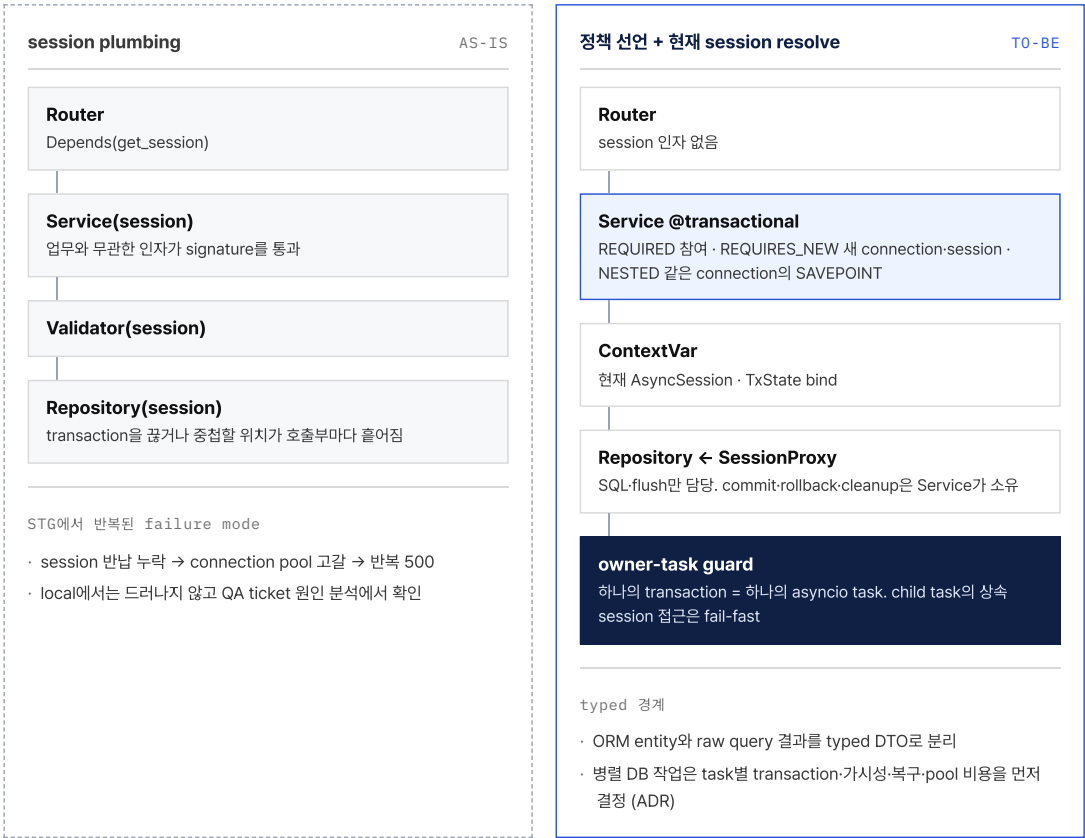
같은 트랜잭션 안에서 여러 task가 AsyncSession을 공유하지 못하게 했습니다. 병렬 DB 작업이 필요하면 task별 트랜잭션, 데이터 가시성, 실패 복구와 connection 비용을 먼저 결정하도록 ADR-runbook에 기록했습니다.

child task의 동일 session 접근을 즉시 실패시키고, CanceledError가 발생하면 rollback과 connection cleanup이 실행되는지 통합 테스트로 확인했습니다. propagation별 commit-rollback과 SAVEPOINT 동작도 함께 고정했습니다.

설계

transaction-session 경계 — session 인자 전달을 없애고 Service가 정책을 선언

판단 비교 · AS-IS / TO-BE



불변 조건	commit-rollback-cleanup은 Service의 decorator와 session layer만 소유한다.
버린 대안	Depends(get_session) 자체를 부정하는 것이 아니다. 모든 계층에 session을 흘려보내는 방식을 버렸다.
검증 근거	propagation · isolation · read-only · CanceledError rollback · connection cleanup integration test
관측 결과	full template으로 시작한 신규 사내 프로그램의 STG QA에서 session 미반납-pool 고갈 재관측 없음

왼쪽은 인계받은 코드에서 반복되던 구조, 오른쪽은 조직 template의 기본값이다.

AI 글 품질 판정 3층 — 프롬프트와 게이트는 바닥만 높인다. 천장은 체크리스트가 못 만든다

구조 · 판정 층



규율	언제	하는 일	통과	어기면	기록
축 먼저	실험 시작 전	어느 축을 올리려는 실험인지 먼저 말한다	축이 있는 실험	실험하지 않는다	축별 점수 이력
근거 붙이기	채점할 때	축마다 왜 이 점수인지, 뭘 고치면 오르는지, 그 근거	게이트 수차·원문 사례·실측 대조가 붙은 점수	점수만 있는 채점 무효	자가 채점 / 합의 채점 구분
기준 재실측	기준값을 쓸 때	기준이 실측인지 자사 출력의 되먹임인지 확인	실측 corpus 기준	자기 과교정을 목표로 삼는 순환	기준 변경 시 이전 점수와 직접 비교 금지
반증 로그	접근이 실패했을 때	지우지 않고 「다시 시도하지 말 것」에 남긴다	실험 범위가 좁아진다	같은 실패 반복	반증된 규칙은 자기 규칙이라도 제거

- 불변 조건**
아래층을 못 넘으면 위층을 묻지 않는다. 자동화는 바닥에만 돈다.
- 버린 대안**
글자 수 상한 지시 (형태를 바꿔도 과교정을 유발 → 분포·총량 비례로 전환) · 자사 출력 값을 기준으로 삼는 것 (재실측으로 발견해 교정)
- 검증 근거**
6축 점수 이력 · 소표본에서 대량 corpus로 측정 확대하며 유지·기각 결론 구분 · 판정 책임을 writer에서 planner로 옮긴 실측 (정보를 가진 자리로 책임 이동)

아래층을 통과하지 못하면 위층을 묻지 않는다. 자동화가 닿는 층과 닿지 않는 층을 처음부터 갈라 설계했다.

설계

prototype backend 재구축 — 돌아가는 기능을 왜 다시 만드는가에 대한 판단과 실행 순서

판단 비교 · 실행 순서

부분 수정 유지

대안 A

기능은 돌아간다
새로 만드는 비용 0

도메인 간 의존성 얹힘
회원 로직 수정이 AI 생성 중단으로 이어진 실사례

QA 티켓이 닫힌 뒤 같은 영역에서 다른 형태로 재발
수정할수록 누적

AI 모듈 확장이 막힌다
확장 지점이 없음

이 대안이 맞는 조건

- 서비스가 이미 크고 이관 비용이 재발 비용보다 클 때
- 확장 계획이 없을 때

병렬 재구축 · frontend 유지

채택

지금 가장 싼 시점
서비스 규모가 작고 사용자 운영 전

backend만 FastAPI로 분리
Next.js frontend는 유지. 화면 회귀 없음

규칙을 먼저 세팅
디자인 패턴 · 컴포넌트 설계 · 하네스 → 그 위에서 AI와 협업

파악 → 기능 정의 → 재구축 → FE 호출 전환
기존 서비스는 그동안 그대로 운영

cutover 뒤 관측
QA reopen을 같은 정의로 계속 측정

선택한 근거

- 문제 누적 속도
- AI 확장성
- 하네스 기반 이관 속도 (작업 시간 기준)



불변 조건	돌아가는 서비스를 멈추지 않는다. 재구축은 규칙(패턴·계층·하네스)을 먼저 세우고 그 위에서 한다.
버린 대안	부분 수정 유지 (재발 누적) · frontend까지 동시 교체 (회귀 범위 확대) · 규칙 없이 AI로 바로 생성
검증 근거	commit 실측 순서 · backend commit 대다수 본인 author · Jira 전이 전수 분석
관측 결과	QA 버그 재발률(해결 대비 reopen) 37% → 11%, 재발 발생 일평균 약 94% 감소. BE/FE 라벨이 없어 제품 전체 품질 지표로 서술
속도 우선으로 빠르게 검증된 초기 제품을 운영 단계로 전환한 판단이다. 전임 작업의 평화가 아니라 시점과 비용의 비교다.	

여러 화면에 흩어진 Thready 기능을 대화 하나로 제어하는 방식을 검증했습니다.

사용자가 기능의 위치와 실행 순서를 외우는 대신, 원하는 작업을 말하면 Agent가 필요한 Thready 기능을 선택하고 결과를 대화로 이어서 보여주는 독립 프로토타입을 만들었습니다.

이 버전에서 보는 지점

생성 파이프라인에서 글 유형 분기 판정을 writer에 뒀을 때 18건 전부 미발동한 문제를 판정 위치를 planner로 옮겨 풀었고, 15노드 legacy 그래프를 publishable 단일 파이프라인으로 축소한 뒤 SDK 기반 단일 agent 엔진을 같은 guard-repair 코드로 병존시켜 실 파이프라인에서 실행 시간을 비교했습니다(agent 엔진 43초 vs 그래프 294초). 대화형 제어는 multi-agent 대신 하나의 planner-executor로 두고, planner는 typed plan만 만들고 실행 권한은 capability registry가 확인한 뒤 dispatch합니다. 예약·발행·삭제 같은 변경은 다음 turn의 typed confirmation을 통과할 때만 실행되고, receipt 기반 idempotency로 중복 실행을 막았습니다.

확인할 근거

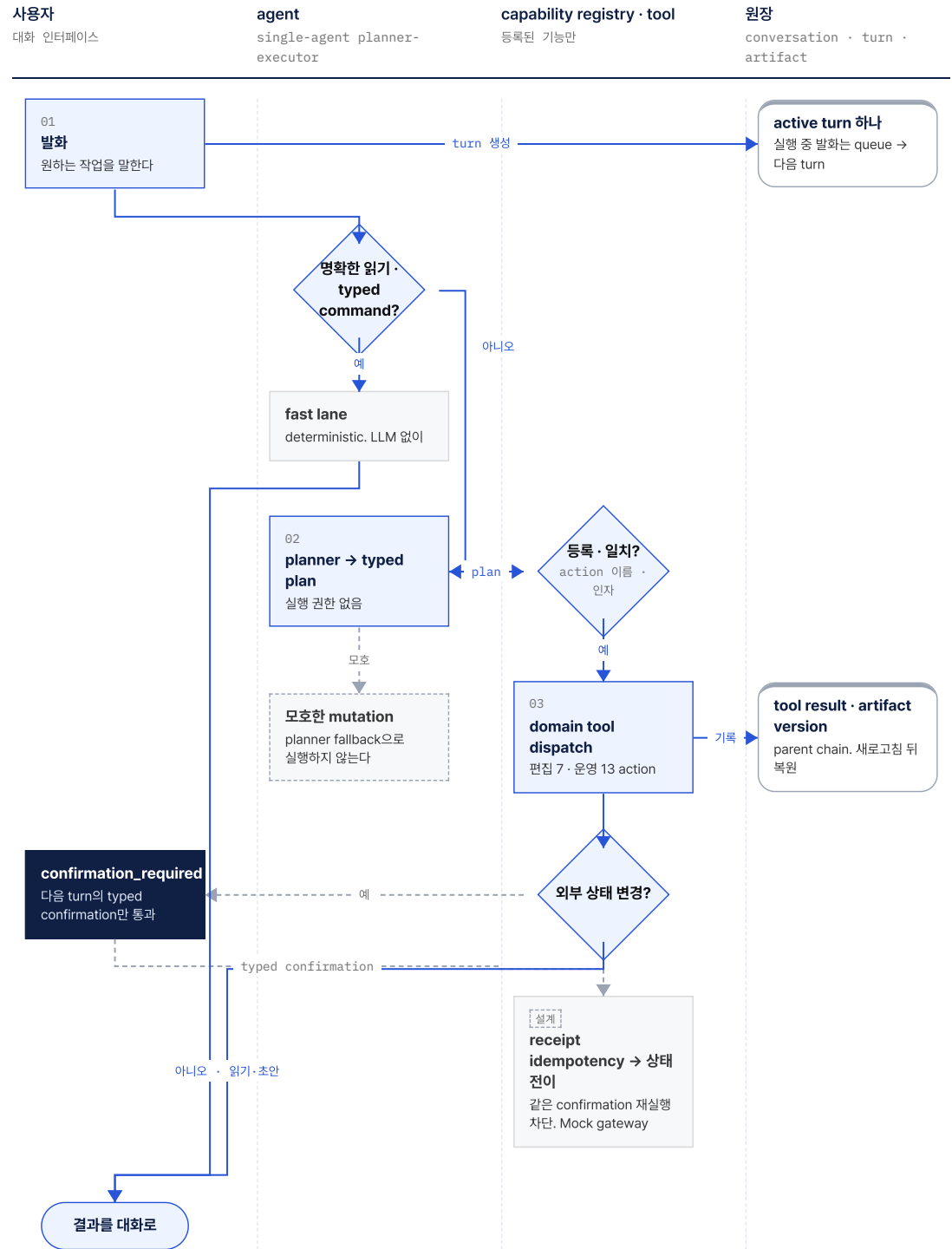
콘텐츠 제작·계정 및 게시물 관리·성과 조화를 하나의 대화 흐름으로 구성

실행 계획과 등록 capability의 실행 권한 분리

대화·실행 요청·도구 결과·결과물 버전 기록

사용자 확인과 중복 실행 방지를 거치는 Mock 상태 변경

운영 Agent가 아닌 독립 프로토타입으로 범위 명시



불변 조건	planner는 실행 권한을 갖지 않는다. 등록되지 않은 action은 실행되지 않는다. 외부 상태 변경은 typed confirmation과 receipt를 통과할 때만 일어난다.
버린 대안	multi-agent orchestration (범위 대비 복잡) · 최근 message 개수 절단 (token-aware context selection + append-only compaction snapshot으로 대체)
검증 근거	대화-queue-artifact · 편집 action · activity · capability · Mock operation 각 test 통과. 운영용 durable worker와 process restart recovery는 미구현 범위

독립 prototype이다. 편집 기능은 기존 writer를 재사용했고, 계정-게시물 운영 action은 Mock gateway에서 실행 흐름과 안전 경계만 검증했다. 실제 예약·발행·삭제는 수행하지 않았다.

Knowledge Gateway	Scout · Friend	Creator · Guard	사용자
계정 범위 · 권한 · 승인 version 필터	소재(설계) · 정체성(구현)	생성 · 검수	

01 [설계]

context pack 반환

목적·주제·최신성·권한에 맞춘
작은 pack. 모든 자료를
prompt에 붙이지 않는다

search_evidence · get_identity · get_learning

02 [설계]

Scout → material pack

이번 글의 소재 · 근거 citation ·
최신성

03 [구현]

Friend → direction pack

brand wizard: extract → build
→ judge. 정체성·voice·관점을
제안하고 사람 confirm 뒤 반영

[구현]

사람 confirm

개인화 결과 확정

get_generation_context

identity · evidence · learning · strategy · policy의 필요한 version

04 [구현]

Creator → draft

planner/writer 생성 · 사용
citation · context version ·
generation trace

05 [구현]

Guard → 통과 / 수정 요청

referee prompt
check(evaluate →
recommend) · writer judge ·
검수 이력. 콘텐츠 방향·개인
memory는 바꾸지 않음

초안 + 발행 시점 제안

scheduler: 계정 성과 feature → candidate plan (LLM은 사유만)

06 [구현]

즉시 발행 · 예약 · 안 씀

사람 확인 없는 자동 발행은 없음

학습 루프

[설계]

Threads 발행 → 반응

선택·수정·반응이 학습 데이터

judge · 성과

[설계]

learning change request

trace의 평가 결과. wiki를 직접
덮지 않는다

앞단 운영자 검토로 복귀

[설계]

승인분만 다음 projection

장기 memory 자동 승격 없음

역할	읽는 정보 · tool	만드는 결과	다음 역할로	직접 바꾸지 않는 것	구현 상태
설계 Scout	search_evidence — 주제·최신성·계정 범위의 승인 evidence	material pack (소재 · citation · 최신성)	Friend · Creator 입력	raw-extract 처리, identity-strategy-policy	자동 소재화 계약만 선언. 트리거·주기·승인 흐름 미확정
구현 Friend	get_identity · get_learning · search_evidence	direction pack (profile · voice · 관점 · 의도)	사람 confirm → Creator	raw 원문, 공통 strategy, 장기 memory 직접 승격	brand wizard graph(extract → build → judge)로 구현. gateway형 tool 계약은 설계
구현 Creator	get_generation_context — 필요한 version만 묶은 context pack	draft · 사용 citation · generation trace	Guard	wiki 장기 지식, policy, 발행 승인	writer 역할 객체 + repair/judge 루프로 구현
구현 Guard	draft · 적용 policy · evidence citation	통과 / 수정 요청과 이유	사용자 선택	콘텐츠 방향 · 개인 memory	결정적 출구 검증 + writer judge + referee prompt check(evaluate → recommend → finalize)

불변 조건	agent는 gateway가 돌려주는 승인 version만 읽는다. 평가 결과는 wiki를 직접 덮지 않고 앞단 검토로 되돌아간다. 발행 여부는 사용자가 정한다.
버린 대안	production Scouter를 먼저 만드는 것 (동의 범위·수집 주기·승인 흐름이 미확정) · 사람 확인 없는 자동 발행 · 장기 memory 자동 승격
검증 근거	설계안의 역할 계약 표 + 운영 코드의 friend(brand wizard) · writer · referee(prompt check) · scheduler(발행 시점) 역할 객체와 예약· 발행 화면. Scout의 production 구현은 주장하지 않는다

Scout는 설계 단계, Friend(brand wizard)·Creator(writer)·Guard(referee-judge)·발행 시점 제안(scheduler)·사용자 발행 선택은 운영 구현이 확인된 요소다.
사람 게이트는 세 곳: 운영자 승인, Friend 개인화 confirm, 사용자 발행 선택.

아이디어를 팀과 실제 고객이 구독하는 제품으로 만들고, 운영에 필요한 시스템을 직접 구축했습니다.

고객이 돈을 내는 이유를 기획·QA·마케팅과 함께 구체화하고, 제품 판단부터 출시·운영까지 이끌었습니다. 제품에 필요한 Next.js 핵심 흐름을 직접 구현하고, 빠른 기능 검증 중심의 초기 백엔드를 인계받아 팀이 운영할 수 있는 FastAPI 구조로 재구축했습니다. 이어 제품 기준 데이터와 AI 실행 상태의 관리 책임을 나눴습니다. 팀과 함께 **구독료를 내는 고객이 쓰는 제품**으로 만들었습니다.

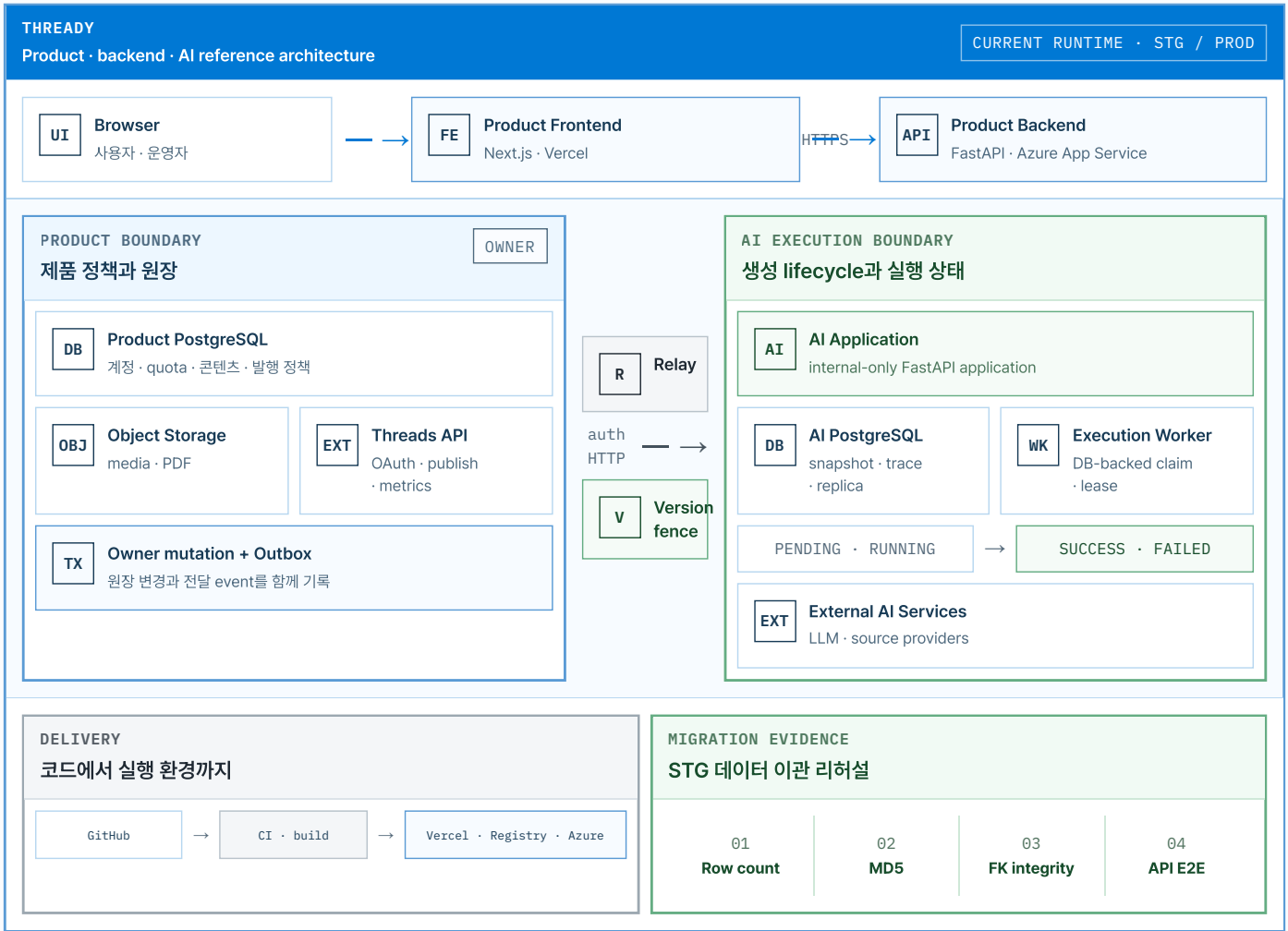
이 버전에서 보는 지점

Threads 글 제작의 반복을 제품 흐름으로 바꾸고, AI는 자료 정리·초안·1차 검수를 맡고 최종 수정·예약·발행 판단은 사람이 하도록 설계한 판단, 그리고 품질 판정을 자동 게이트·실측 분포·사람 판정 3층으로 나눈 체계를 봅니다. 프롬프트 기준값이 자사 출력을 되먹이던 순환을 재실측으로 발견해 기준을 교정했습니다.

CURRENT · SUBSCRIPTION 유료 제품 운영 구독 결제 · 제품 운영	NEXT · ADVERTISING 광고 수익 모델 검증 적용 시작 · 운영 데이터 수집 중
--	---

제품 운영과 실제 실행 환경을 한 구조로 연결했습니다.

PRODUCT · RUNTIME ARCHITECTURE



리드 제품 운영 · 우선순위 · 품질 · QA · 릴리스	직접 구현 제품 핵심 흐름 · Next.js · FastAPI · 데이터 · AI	팀 협업 기획 · QA · 마케팅 · 디자인
---	---	------------------------------------

한 모델의 장애가 전체 생성 기능 중단으로 번지지 않게 운영했습니다.

외부 AI 장애 대응

감지 Sentry 알림 반복되는 외부 AI 5xx를 운영자가 확인	격리 문제 모델 일시 제외 오류가 반복되는 모델을 사용자 선택지에서 제거	작업 지속 정상 모델 선택 서비스 중인 다른 모델로 생성을 계속 진행
--	--	--

5xx는 재시도 가능한 실패로 분류하고 최대 시도 뒤 최종 실패 상태와 사용자 재시도 경로를 남겼습니다. 당시 사용자·트래픽 규모에서는 복잡한 자동 복구보다 운영자가 문제 모델만 격리하는 방식이 적절하다고 판단했습니다.

공개 콘텐츠 분석을 제품 판단과 생성·평가 실험에 연결했습니다.

콘텐츠 데이터

라벨링 도구

Threads 공개 게시글과 작성자의 이어쓰기를 독립된 라벨링 영역으로 정리했습니다. 같은 자료를 다시 가져와도 중복되지 않는 가져오기 도구와 API·평가 화면을 함께 만들었습니다.

잘되는 글의 기준

최근 1년 내 게시된 콘텐츠를 중심으로 첫 문장·구조·어체·마무리의 관측 기준을 만들었습니다. 이 기준은 글 생성 프롬프트와 평가 실험에 반영해 결과를 비교했습니다.

인계받은 초기 백엔드를 팀이 운영할 수 있는 FastAPI 구조로 재구축했습니다.

백엔드 재구축

무엇이 문제였나 · 인계받은 초기 백엔드는 빠른 기능 검증 중심으로 만들어져 있었습니다. 기능은 돌아갔지만 어떤 코드가 무엇을 책임지는지, 수정 영향이 어디까지 퍼지는지 설명하기 어려웠습니다.

어떻게 풀었나 · 기존 API와 기능을 먼저 목록으로 만들고, 현재 동작을 확인할 테스트와 검증 기준을 세웠습니다. 프론트엔드는 그대로 둔 채 새 백엔드를 옆에서 만들었고, AI는 코드 분석과 반복 구현에 활용했습니다. 아키텍처와 검증 기준, 작업 범위, 전환 시점은 직접 판단했습니다.

무엇이 달라졌나 · 기존 백엔드와 새 백엔드의 응답을 비교한 뒤 전환했습니다. 전환 전후 같은 기준으로 비교했을 때, 해결된 QA 항목의 재오픈 비율은 26%p 낮아졌습니다. 이후 배포와 운영도 계속 맡았습니다.

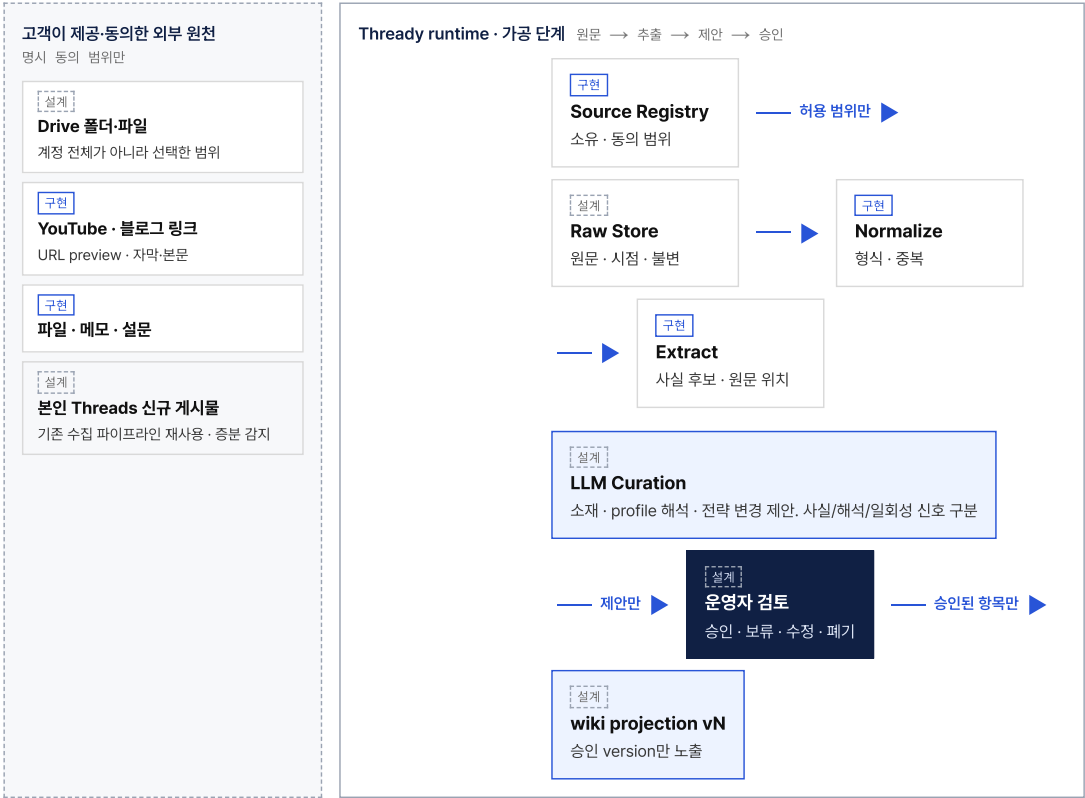


제품 원장과 AI 실행을 나누고, 중복·지연·역순 전달에도 최신 상태로 수렴하도록 설계했습니다.

BE-AI 분리 설계

01 · OWNER TX	02 · RELAY	03 · VERSION FENCE	04 · AI STATE
원장 변경 + Outbox	retry	최신 상태 보호	별도 application · DB
제품 원장 변경과 전달할 event를 같은 transaction에 기록	일시적인 전달 실패는 원장 transaction과 분리해 다시 시도	늦거나 역순인 변경이 더 최신 상태를 덮지 않도록 차단	생성 lifecycle과 실행 상태는 AI 경계가 소유

데이터 이전 검증	STG의 생성 이력 2,616건·품질 기록 795건·실행 추적 7,111건을 로컬에 복원해 이관 리허설을 진행했습니다. 이후 행 수와 MD5 fingerprint, 참조 누락, API 동작을 확인했습니다.
-----------	--



지식 항목 상태별 처리

상태	진입 조건	이 상태에서 하는 일	다음 전이	철회·정정 시	관측·개입
raw	동의된 원천에서 수집	원문·시점·checksum 보관, 원칙적으로 불변	extract 생성	권한 철회 → source 비활성, downstream 재검토	마지막 수집 시점
extract	raw 존재	텍스트·사실 후보·원문 위치, 추출기 개선 시 재생성 가능	LLM curation 입력	해당 raw 기준으로 다시 생성	추출물마다 raw 참조
제안	LLM curation	변경 대상·근거·적용 범위 명시	운영자 승인 → projection	보류 · 수정 · 폐기	승인 대기 수
projection vN	승인	역할별 read view, 승인 version만 노출	agent가 gateway로 읽음	영향받는 항목만 되돌리거나 폐기	version별 사용 trace
trace	생성 완료	읽은 version·근거·결과·평가, append-only	learning change request → 검토로 복귀	원문 철회 시 접근 차단 · 영향 범위 표시	회고 · 재현

- 불변 조건

raw 원문은 wiki에 바로 쓰지 않는다. wiki에는 승인된 version만 올라간다. policy는 LLM이 바꾸지 않는다.
- 버린 대안

모든 자료를 prompt에 붙이는 방식 · Drive를 유일한 Raw Store로 두고 생성 시마다 외부 원천을 읽는 방식 · 원문·추출·해석을 한 저장소에 섞는 방식
- 검증 근거

설계안(exploration) + 운영 코드의 콘텐츠 가져오기·URL preview·source ownership 검증, 파일럿 최소 범위는 운영자 등록 → 추출 → 제안 → 승인 → 초안·trace → learning 후보

「구현」은 운영 Thredy 코드에서 확인된 요소, 「설계」는 설계 문서 단계다. 이 설계안은 exploration이며 제품 계약이 아니다. 설계와 콘텐츠 가져오기·생성·검수 workflow 구현은 직접 담당했다.

실패한 작업은 다시 돌리고, 실시간 상담은 빠르게 반응하면서도 엉뚱한 발화를 덮지 않게 만들었습니다.

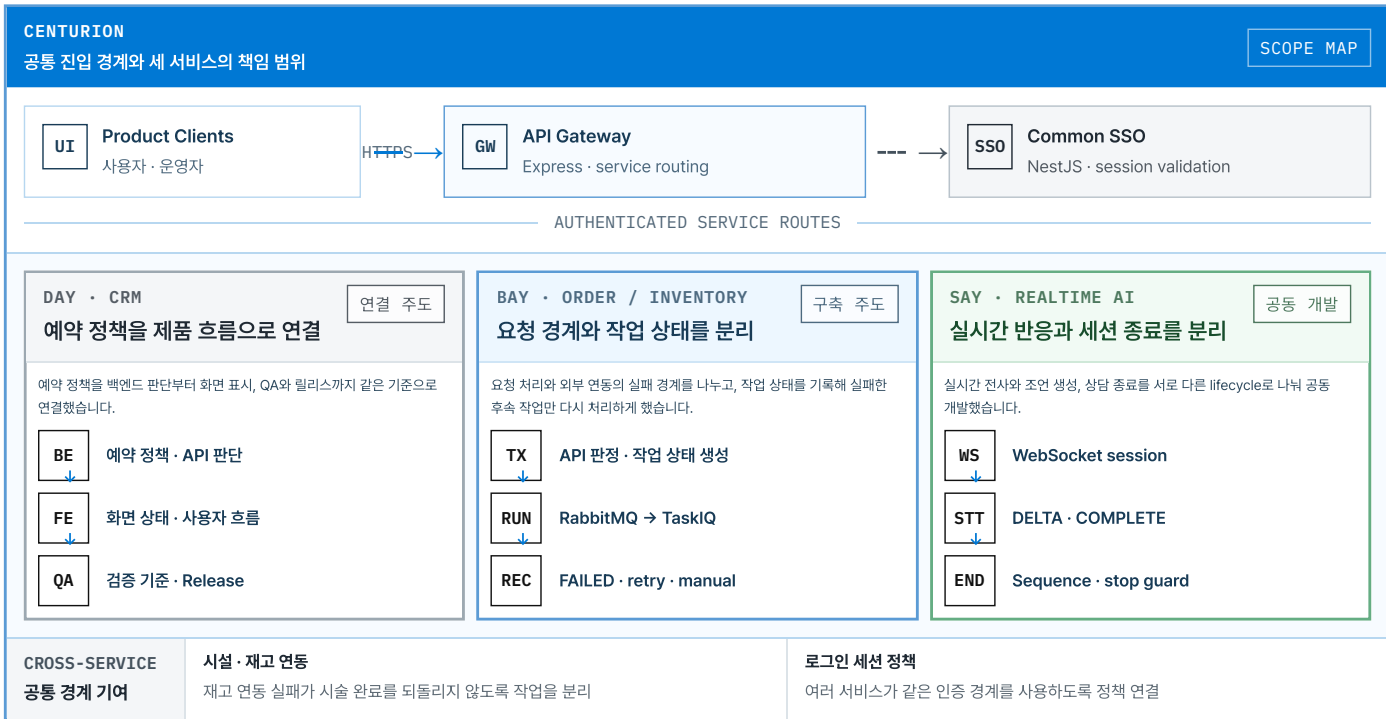
병원 운영 플랫폼에는 예약·주문·재고처럼 즉시 저장해야 하는 업무, 실패하면 다시 실행해야 하는 후속 작업, 연결을 유지해야 하는 실시간 상담이 함께 있습니다. 하나의 패턴을 모든 서비스에 강요하지 않고, **각 서비스가 저장할 데이터와 복구할 작업**을 따로 정했습니다.

이 버전에서 보는 지점

이 사례에서는 실시간 상담 backend를 봅니다. 실시간 STT가 내보내는 부분 전사와 완성 전사, 비동기로 뒤늦게 도착하는 보정 결과가 같은 WebSocket 세션 안에서 섞여 늦은 보정이 다른 turn을 덮거나 중복 event가 상태를 흔들었습니다. 부분 전사는 domain keyword 우선 판정에, 완성 전사는 문맥 판단·저장에 쓰도록 역할을 나누고 부분·완성·보정을 같은 sequence로 묶어 순서로 정합을 보장한 판단, 중복 event를 task cancellation·debounce·retry·turn·state guard로 제어한 구현, 그리고 외부 모델 세션이 끊겨도 발화 순서가 유지되는 경계와 provider 후보 비교용 WER·CER·keyword retention·latency benchmark를 봅니다.

예약 정책은 출시 기준까지 연결하고, 실패한 작업은 다시 처리할 수 있게, 실시간 상담은 재연결·종료 경계를 검증했습니다.

CONTRIBUTION MAP



서비스별로 해결한 문제

구현 내용

주문·재고

구축 주도

요청 처리와 외부 연동의 실패 경계를 나누고, 명시적인 작업 상태·재시도·최종 실패·수동 복구 경계를 만들었습니다.

실시간 AI 상담

공동 개발

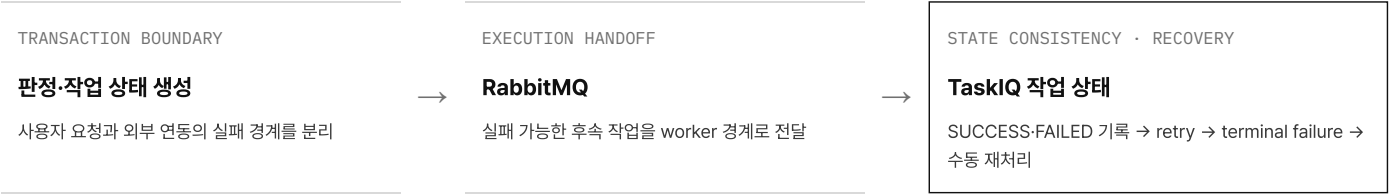
DELTA→COMPLETE 전사 흐름과 session lifecycle을 나누고, 도메인 키워드 우선 판정·sequence guard·재연결 정리를 구현했습니다.

DAY 예약 정책	연결 주도	예약 정책을 백엔드 판단부터 프론트엔드 표시·QA·릴리스까지 같은 기준으로 연결했습니다.
시설·재고·공통 인증	주요 기능 개발	재고 연동이 실패해도 시술 완료가 취소되지 않도록 작업을 분리했습니다.

외부 연동 실패가 사용자 요청 전체를 되돌리지 않게 했습니다.

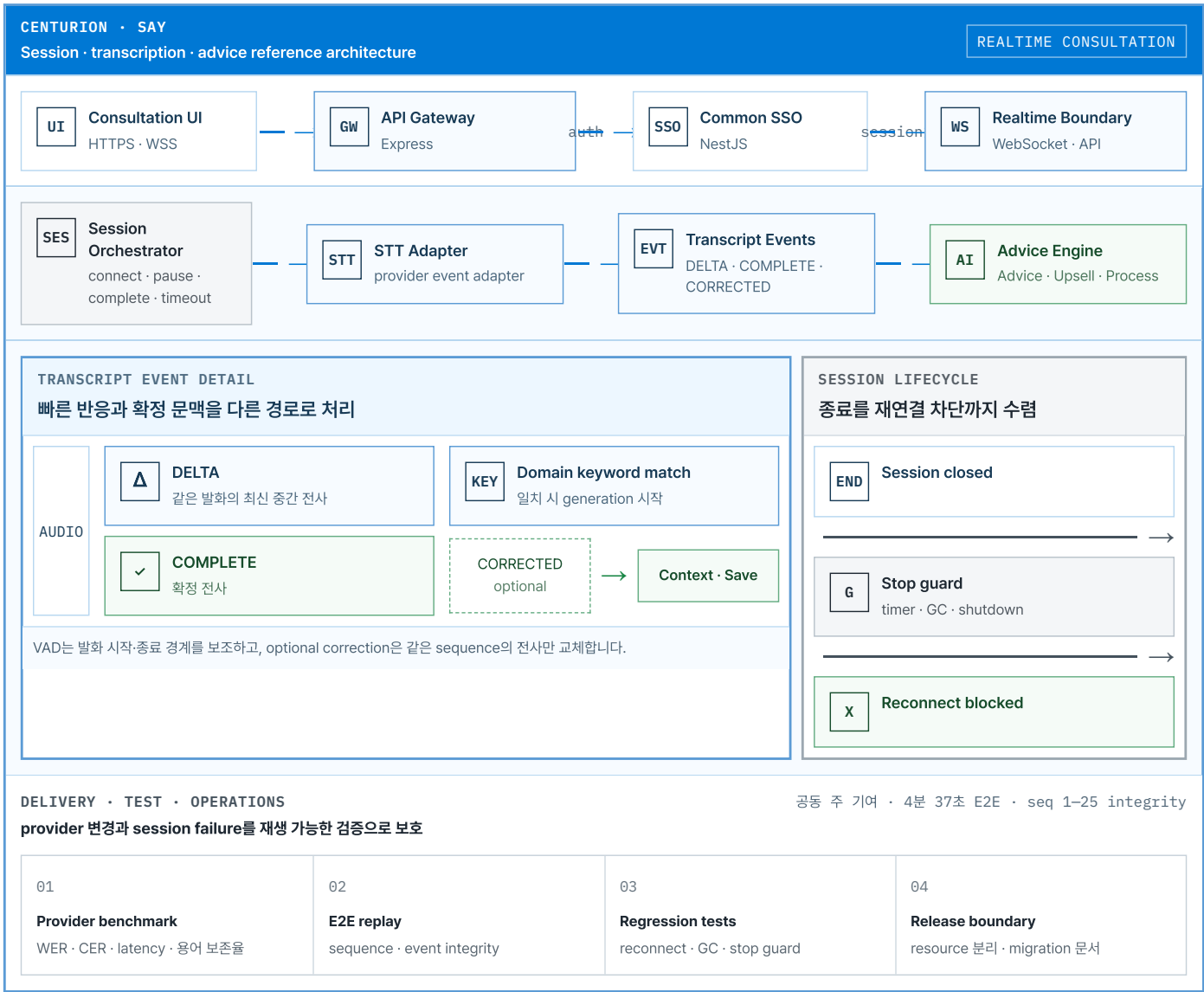
TRANSACTION · STATE · RECOVERY

기존에도 Celery로 API 밖의 작업을 처리하고 있었습니다. 전환의 이유는 비동기 분리 자체가 아니라, async FastAPI 코드베이스와 worker의 실행 모델을 맞추기 위함이었습니다. RabbitMQ는 유지하고 TaskIQ로 전환하며 상태·재시도·최종 실패·수동 재발송을 주문 운영 흐름에 연결했습니다.



설계 원칙 · 실패 상태를 기록으로 남겨 사용자 요청 전체가 아니라 실패한 후속 작업만 다시 실행합니다.

Gateway와 SSO를 거친 상담 연결은 WebSocket session orchestrator가 관리합니다. STT에서 나온 중간 전사 DELTA는 도메인 키워드를 먼저 확인하는 데 사용하고, COMPLETE는 확정 문맥과 저장에 사용합니다. 다만 COMPLETE의 도착 순서가 보장되지 않아 DELTA·COMPLETE·CORRECTED를 같은 sequence로 연결했고, 늦은 보정이 다른 발화를 덮지 않게 했습니다. 상담 종료 뒤에는 timer·GC·shutdown의 정리 책임을 stop guard로 모았습니다.



SESSION LIFECYCLE

상담이 끝난 뒤 다시 연결되던 세션을 멈췄습니다.

WebSocket 종료 뒤 reconnect timer가 남아 외부 AI 연결이 다시 살아나는 경로를 재현했습니다. pause-complete-timeout-GC-shutdown의 정리 책임을 나누고, 재연결 진입 전과 backoff 이후에 stop guard를 두었습니다.

reconnect race 8개 · GC TTL 5개 – 13개 회귀 시나리오로 고정



구성 요소별 책임 — 각 부품은 자기 실패만 안다

구성 요소	받는 것	책임	내보내는 것	실패했을 때	모르는 것
Slicer	연속 오디오	window가 stagger보다 길게 잘라, 어느 순간에도 두 세션 이상이 같은 구간을 듣게 한다	검치는 window	없음 (결정적)	세션 상태
세션 풀	window	동시 N개 세션. 각 세션은 자기 window만 담당하고 끝나면 다음 window로 순환	세션별 전사 조각	세션 종료:timeout은 그 window 하나만 읽는다. 풀은 계속 돈다	다른 세션의 진행
세션별 버퍼	전사 조각	세션 안에서 순서를 지키고, 문장 종결 부호에서 COMPLETE로 자른다. 재연결 시 버퍼를 비운다	COMPLETE + sequence	부분 문장은 다음 조각과 이어 붙인다	다른 세션의 버퍼
병합기	여러 세션의 COMPLETE	세션이 아니라 sequence 기준으로 하나의 흐름으로 정렬한다	순서 있는 전사 스트림	빠진 sequence는 기다리지 않고 다음으로 넘긴다 (실시간 우선)	어느 세션이 죽었는지
하류 트리거	COMPLETE	advice-추천 판단을 시작한다	상답 조언	COMPLETE가 늦으면 늦게 시작할 뿐, 멈추지 않는다	세션 풀의 존재

- 불변 조건

세션 하나가 죽어도 전사는 멈추지 않는다. 검침은 window > stagger에서 나오고, 순서는 세션이 아니라 sequence가 정한다.
- 버린 대안

단일 세션 + 재연결 (끊긴 구간이 그대로 유실되고, 재연결 타이머 경합이 약 1분 시점 응답 중단을 만들었다) · 모델 교체를 기다리며 기능을 세워 두는 것
- 검증 근거

개발 test로 풀 크기와 window-stagger를 정한 뒤 dev/stg에 적용. 동시 세션 수만큼 비용이 늘고 첫 응답이 늦어지는 손실은 문서로 명시. 모델 교체 뒤 레거시 provider와 함께 제거

외부 실시간 음성 모델의 세션이 통제할 수 없는 시점에 종료되던 문제를, 모델 교체가 오기 전까지 구조로 흡수했다. 빗금은 두 세션이 같은 구간을 듣는 겹침이고, 점선은 끊긴 세션이다. 모델 교체 뒤 이 우회 구조는 걷어냈다.

